

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Построение и анализ алгоритмов»

Вариант: Вар. 1. МВиГ: Алгоритм Литтла. Приближённый алгоритм: 2-
приближение по МОД (АДО МОД). Замечание к варианту 1. АДО МОД
является 2-приближением только для евклидовой матрицы. Начинать обход
МОД со стартовой вершины

Студент гр. 4343

Зеров М.М.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Разработайте программу, которая решает задачу коммивояжера при помощи 2-приближенного алгоритма. В данной постановке задачи нужно вернуться в исходную вершину после прохождения всех остальных вершин. При обходе остовного дерева (MST) необходимо идти по минимальному допустимому ребру из текущего. Каждая вершина в графе обозначается неотрицательным числом, начиная с 0, каждое ребро имеет неотрицательный вес. В графе нет рёбер из вершины в саму себя, в матрице весов на месте таких отсутствующих рёбер стоит значение -1.

Пример входных данных

Входные данные: 2

Выходные данные: -1 18.97 22.36 19.42 3.61

18.97 -1 35.61 38.01 17.0

22.36 35.61 -1 16.28 21.19

19.42 38.01 16.28 -1 21.02

3.61 17.0 21.19 21.02 -1

В первой строке указывается начальная вершина.

Далее идёт матрица весов.

В качестве выходных данных необходимо представить длину пути, полученного при помощи алгоритма. Следующей строкой необходимо представить путь, в котором перечислены вершины, по которым необходимо пройти от начальной вершины. Для приведённых в примере входных данных ответом будет

91.92

2 3 0 4 1 2

Тестовые данные

№ Теста

1

Входные данные

10

-1 9.22 23.32 8.49 16.76 30.0 26.02 13.89 3.61 13.15 21.4 8.06 4.12
9.22 -1 32.2 17.69 25.5 39.2 33.38 13.93 12.73 21.02 30.48 5.1 11.66
23.32 32.2 -1 15.23 15.52 12.17 25.24 27.29 19.72 12.21 3.16 29.0 20.62
8.49 17.69 15.23 -1 10.05 21.63 21.19 18.03 5.0 8.06 13.04 15.65 7.28
16.76 25.5 15.52 10.05 -1 15.26 11.66 28.07 14.14 16.12 12.37 24.74 17.03
30.0 39.2 12.17 21.63 15.26 -1 19.1 37.64 26.63 22.56 11.05 37.22 28.65
26.02 33.38 25.24 21.19 11.66 19.1 -1 38.83 24.33 27.78 22.2 34.0 27.46
13.89 13.93 27.29 18.03 28.07 37.64 38.83 -1 14.56 15.23 26.93 8.94 11.4
3.61 12.73 19.72 5.0 14.14 26.63 24.33 14.56 -1 10.0 17.8 10.77 3.16
13.15 21.02 12.21 8.06 16.12 22.56 27.78 15.23 10.0 -1 11.7 17.2 9.49
21.4 30.48 3.16 13.04 12.37 11.05 22.2 26.93 17.8 11.7 -1 27.66 19.1
8.06 5.1 29.0 15.65 24.74 37.22 34.0 8.94 10.77 17.2 27.66 -1 8.6
4.12 11.66 20.62 7.28 17.03 28.65 27.46 11.4 3.16 9.49 19.1 8.6 -1

Выходные данные

147.25

10 2 5 9 3 8 12 0 11 1 7 4 6 10

Описание алгоритма:

Матрица Литла

Шаг 1: Инициализация

$cost[i][j]$ — матрица весов, -1 заменяется на INF (ребра нет).

$bestCost = +inf$, $bestPath = \text{пусто}$.

Если запускается вместе с АДО МОД, $bestCost$ инициализируется верхней оценкой из МОД (ускоряет отсечения).

Шаг 2: Рекурсивный DFS по дереву перебора

На каждом шаге:

$path$ — текущий частичный маршрут (от стартовой вершины);

$curCost$ — его стоимость;

rem — множество ещё не посещённых вершин.

Шаг 3: Нижняя оценка (редукция матрицы Литтла)

Строится матрица m размера $(|rem|+1) \times (|rem|+1)$:

строка 0 — переходы из текущей вершины cur ;

строки 1..k — переходы из $rem[i-1]$;

столбец $|rem|$ — возврат в стартовую вершину.

Редукция:

для каждой строки вычитается её минимум;

для каждого столбца вычитается его минимум;

сумма вычтенных минимумов = нижняя оценка.

$lb = curCost + \text{littleReduce}(rem, cur, START)$

Шаг 4: Отсечение

Если $lb > bestCost + EPS$, ветка отбрасывается — она не может привести к решению лучше уже найденного.

Шаг 5: Ветвление

Для каждой вершины nxt из rem (если $cost[cur][nxt]$ конечно):

```
path.push_back(nxt);  
dfs(nxt, visited, path, curCost + cost[cur][nxt]);  
path.pop_back();
```

Шаг 6: Лист дерева

Когда $path.size() == N$:

```
если  $cost[cur][START]$  конечно:  
    total = curCost + cost[cur][START];  
    если total < bestCost:  
        bestCost = total; bestPath = path;  
    вывод "[Улучшение] ..."
```

Шаг 7: Завершение

Возврат $bestPath$ и $bestCost$ — точное оптимальное решение.

2-приближение по МОД

Шаг 1: Построение МОД (алгоритм Прима)

```
key[START] = 0;
```

На каждой итерации:

```
выбирается вершина u с минимальным key среди ещё не включённых;  
в MST добавляется ребро parent[u] - u;  
обновляются ключи соседей.
```

Если после N итераций охвачены не все вершины — граф несвязный.

Шаг 2: Сортировка рёбер МОД

Для каждой вершины рёбра сортируются по весу — это соответствует требованию "идти по минимальному допустимому ребру из текущего".

Шаг 3: Preorder-обход дерева (DFS)

Начиная со стартовой вершины, рекурсивно обходим дерево.

Получаем последовательность `tour`, в которой каждая вершина встречается ровно один раз (preorder).

Шаг 4: Замыкание цикла

$$\text{total} = \sum \text{cost}[\text{tour}[i]][\text{tour}[i+1]] + \text{cost}[\text{tour}.\text{back()}][\text{START}].$$

Если какое-то ребро отсутствует (INF) — метод отказывается считать оценку и возвращает INF.

Шаг 5: Возврат результата

`tour` — приближённый маршрут,

`total` — его стоимость (верхняя оценка оптимума).

Оценка сложности:

Обозначения:

N — количество городов.

Метод 1 (Алгоритм Литтла):

Число листьев дерева перебора в худшем случае: $(N-1)!$

На каждом узле строится матрица Литтла размера $\leq N \times N$
и выполняется редукция за $O(N^2)$.

Итоговая худшая сложность: $O(N! \cdot N^2)$.

Метод 2 (АДО МОД):

Preorder-обход: $O(N)$.

Подсчёт стоимости: $O(N)$.

Итоговая сложность: $O(N^2)$.

Тесты:

ТЕСТЫ

```
[=====] Running 12 tests from 1 test suite.  
[-----] Global test environment set-up.  
[-----] 12 tests from TSPTTest  
[ RUN      ] TSPTTest.ExampleN3  
[1] N=3 (пример из условия)
```

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 3x3

Матрица:

-1	1	3
3	-1	1
1	2	-1

МЕТОД 1: 2-приближение по МОД

АДО МОД (preorder-обход): 0 -> 1 -> 2 -> 0

Верхняя оценка: 3.00

МЕТОД 2: Алгоритм Литтла

Оптимальный маршрут: 0 -> 1 -> 2 -> 0

Стоимость: 3.00

РЕЗУЛЬТАТ: Решение найдено!

ОК (маршрут: 0 -> 1 -> 2 0, стоимость: 3.00)

```
[   ОК   ] TSPTTest.ExampleN3 (0 ms)
```

```
[ RUN      ] TSPTTest.ExampleN4
```

```
[2] N=4 (пример из условия)
```

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 4x4

Матрица:

-1	3	4	1
1	-1	3	4
9	2	-1	4
8	9	2	-1

МЕТОД 1: 2-приближение по МОД

АДО МОД (preorder-обход): 0 -> 3 -> 2 -> 1 -> 0

Верхняя оценка: 6.00

МЕТОД 2: Алгоритм Литтла

Оптимальный маршрут: 0 -> 3 -> 2 -> 1 -> 0

Стоимость: 6.00

РЕЗУЛЬТАТ: Решение найдено!

ОК (маршрут: 0 -> 3 -> 2 -> 1 0, стоимость: 6.00)

```
[   ОК   ] TSPTTest.ExampleN4 (0 ms)
```

```
[ RUN      ] TSPTTest.AllWeightsEqual
```

```
[3] Симметричная матрица (N=4, все веса 1)
```

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 4x4

Матрица:

-1	1	1	1
1	-1	1	1
1	1	-1	1
1	1	1	-1

МЕТОД 1: 2-приближение по МОД

АДО МОД: 0 -> 1 -> 2 -> 3 -> 0

Стоимость: 4.00

МЕТОД 2: Алгоритм Литтла

Оптимум: 0 -> 1 -> 2 -> 3 -> 0

Стоимость: 4.00

РЕЗУЛЬТАТ: Решение найдено!

ОК (стоимость: 4.00)

[OK] TSPTTest.AllWeightsEqual (0 ms)

[RUN] TSPTTest.AsymmetricN3

[4] Асимметричная матрица (N=3)

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 3x3

Матрица:

-1	10	5
3	-1	1
2	4	-1

МЕТОД 1: 2-приближение по МОД

АДО МОД: 0 -> 2 -> 1 -> 0

Верхняя оценка: 12.00

МЕТОД 2: Алгоритм Литтла

Оптимум: 0 -> 2 -> 1 -> 0

Стоимость: 12.00

РЕЗУЛЬТАТ: Решение найдено!

ОК (оптимум: 12.00)

[OK] TSPTTest.AsymmetricN3 (0 ms)

[RUN] TSPTTest.N2Trivial

[5] N=2 (две вершины)

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 2x2

Матрица:

-1	7
7	-1

МЕТОД 1: 2-приближение по МОД

АДО МОД: 0 -> 1 -> 0

Стоимость: 14.00

МЕТОД 2: Алгоритм Литтла

Оптимум: 0 -> 1 -> 0

Стоимость: 14.00

РЕЗУЛЬТАТ: Решение найдено!

ОК (стоимость: 14.00)

[OK] TSPTTest.N2Trivial (0 ms)

[RUN] TSPTTest.Disconnected

[6] Несвязный граф (N=4)

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 4x4

Матрица:

-1 5 -1 -1

5 -1 -1 -1

-1 -1 -1 3

-1 -1 3 -1

ПРОВЕРКА ГРАФА:

BFS от вершины 0: посещены {0, 1}

Вершины 2 и 3 недостижимы.

РЕЗУЛЬТАТ: Решение не найдено!

ОК (граф несвязный — обнаружено корректно)

[OK] TSPTTest.Disconnected (0 ms)

[RUN] TSPTTest.VertexDegreeOne

[7] Висячая вершина (N=4)

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 4x4

Матрица:

-1 1 -1 -1

1 -1 1 1

-1 1 -1 1

-1 1 1 -1

ПРОВЕРКА ГРАФА:

Степени вершин: 0 -> 1, 1 -> 3, 2 -> 2, 3 -> 2

Вершина 0 имеет степень 1.

РЕЗУЛЬТАТ: Решение не найдено!

ОК (гамильтонов цикл невозможен)

[OK] TSPTTest.VertexDegreeOne (0 ms)

[RUN] TSPTTest.TwoApproxGuarantee

[8] Проверка гарантии 2-приближения (N=4)

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 4x4

Матрица:

-1 2 9 10

2 -1 6 4

9 6 -1 8

10 4 8 -1

МЕТОД 1: 2-приближение по МОД
АДО МОД: 0 -> 1 -> 2 -> 3 -> 0
Стоимость: 26.00

МЕТОД 2: Алгоритм Литтла
Оптимум: 0 -> 1 -> 3 -> 2 -> 0
Стоимость: 23.00

ПРОВЕРКА ГАРАНТИИ:
МОД = 26.00
2 * ОПТ = 46.00
Условие МОД <= 2*ОПТ: выполнено

РЕЗУЛЬТАТ: Решение найдено!
ОК (гарантия 2-приближения соблюдена)
[OK] TSPTTest.TwoApproxGuarantee (0 ms)
[RUN] TSPTTest.PathLengthCheck
[9] Проверка длины маршрута (N=5)

ВХОДНЫЕ ДАННЫЕ:
Стартовая вершина: 0
Размер: 5x5
Матрица:
-1 10 15 20 25
10 -1 35 25 30
15 35 -1 30 20
20 25 30 -1 15
25 30 20 15 -1

МЕТОД: Алгоритм Литтла
Оптимум: 0 -> 1 -> 3 -> 4 -> 2 -> 0
Стоимость: 85.00

ПРОВЕРКА:
Количество вершин: 5 (ожидалось 5)
Все уникальны: да
Все присутствуют: да

РЕЗУЛЬТАТ: Решение найдено!
ОК (маршрут содержит все вершины ровно один раз)
[OK] TSPTTest.PathLengthCheck (0 ms)
[RUN] TSPTTest.CostConsistency
[10] Проверка стоимости (N=4)

ВХОДНЫЕ ДАННЫЕ:
Стартовая вершина: 0
Размер: 4x4
Матрица:
-1 5 10 15
5 -1 20 25
10 20 -1 30
15 25 30 -1

МЕТОД: Алгоритм Литтла

Оптимум: 0 -> 1 -> 2 -> 3 -> 0
Стоимость (из алгоритма): 70.00

ПРОВЕРКА СТОИМОСТИ (ручной подсчёт):

0 -> 1: 5.00
1 -> 2: 20.00
2 -> 3: 30.00
3 -> 0: 15.00
Итого: 70.00

РЕЗУЛЬТАТ: Стоимость совпадает!

OK (алгоритм: 70.00, ручной подсчёт: 70.00)

[OK] TSPTest.CostConsistency (0 ms)

[RUN] TSPTest.NonZeroStart

[11] Стартовая вершина != 0 (N=4)

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 2

Размер: 4x4

Матрица:

-1	1	3	1
1	-1	1	3
3	1	-1	1
1	3	1	-1

МЕТОД: Алгоритм Литтла

Оптимум: 2 -> 1 -> 0 -> 3 -> 2

Стоимость: 4.00

ПРОВЕРКА:

Первая вершина маршрута: 2 (ожидалось 2)

РЕЗУЛЬТАТ: Решение найдено!

OK (маршрут начинается с вершины 2)

[OK] TSPTest.NonZeroStart (0 ms)

[RUN] TSPTest.MSTReturnsValidTour

[12] АДО МОД возвращает валидный тур (N=4)

ВХОДНЫЕ ДАННЫЕ:

Стартовая вершина: 0

Размер: 4x4

Матрица:

-1	7	12	3
7	-1	5	11
12	5	-1	6
3	11	6	-1

МЕТОД: 2-приближение по МОД

АДО МОД: 0 -> 3 -> 2 -> 1 -> 0

Стоимость: 21.00

ПРОВЕРКА:

Маршрут не пуст: да

Количество вершин: 4 (ожидалось 4)

Первая вершина: 0 (ожидалось 0)

РЕЗУЛЬТАТ: Решение найдено!

ОК (МОД вернул валидный тур)

[ОК] TSPTTest.MSTReturnsValidTour (0 ms)

[-----] 12 tests from TSPTTest (0 ms total)

[-----] Global test environment tear-down

[=====] 12 tests from 1 test suite ran. (0 ms total)

[PASSED] 12 tests.

РЕЗУЛЬТАТЫ

Пройдено: 12

Не пройдено: 0